

# Lab 4 - AI Chat History

Helicopter game (Python / pygame) | Repo: SE-PES1UG24CS614 | AI tool: Claude

## Setup: starting the lab

### Student message

i want to complete this project according to the pdf and fix the bug correctly so give me step by step. (Attached: 08\_helicopter-main.zip, a screen recording of the bug, and the Lab 4 handout PDF.)

### AI response

The AI read the code and the README, which names the bug. It planned one git commit per task, as the lab requires. The student then asked for simpler step-by-step instructions, so the AI gave this workflow:

- Clone the repo SE-PES1UG24CS614 and create a Lab-4/ folder next to Lab1, Lab2 and Lab3.
- Copy the original helicopter code into Lab-4 and commit it: "Lab4: original helicopter code".
- Run python main.py and record the BEFORE video (10 seconds) showing the bug.
- Do Tasks 1 to 4 in one AI chat, one git commit per task.
- Record the AFTER video, export the chat as PDF, and push.

## Task 1: Fix the movement bug

### Problem found

- Holding Up or Down increases vy with no limit, so the helicopter keeps drifting after you reverse.
- There is no bottom boundary check, so the helicopter can leave the screen at the bottom.

### PROMPT SENT TO AI

Holding Up/Down keeps increasing vy with no limit, so reversing direction is laggy, and there's no bottom boundary check. Cap the speed, add drag when no key is pressed, make the opposite key cancel the old speed instantly, and clamp the helicopter at both the top and bottom of the screen.

### What the AI changed (game/helicopter.py)

- Speed is capped with MAX\_SPEED = 6.
- Drag slows the helicopter smoothly when no key is held.
- Pressing the opposite key sets the old speed to zero, so reversing is instant.
- Top and bottom are both clamped using half the helicopter's height.

### How to check it

- Hold Up for 2 seconds, then press Down: it should reverse right away.
- Hold Down: the helicopter stops at the bottom edge and does not leave the screen.

### Git commit

```
git add .
git commit -m "Task 1: fix movement and boundary bug"
```

## Task 2: Collision and GAME OVER

### PROMPT SENT TO AI

Add collision detection between the helicopter and both the top and bottom walls of each obstacle. Touching a wall ends the game and shows a GAME OVER message. Flying through the gap must never end the game. Press R to restart.

### What the AI changed (game\_engine.py, obstacle.py)

- Each obstacle has a top rectangle and a bottom rectangle. The helicopter's rectangle is tested against both.
- The gap between the two rectangles is empty, so flying through it never counts as a hit.
- A hit sets `game_over = True`, which stops updates and shows a GAME OVER banner.
- Pressing R (or Enter) calls `reset()` and starts a fresh game.

### Key code

```
def _hits_obstacle(self, obstacle):
    rect = self.helicopter.get_rect()
    return (rect.colliderect(obstacle.get_top_rect())
            or rect.colliderect(obstacle.get_bottom_rect()))
```

### How to check it

- Touch a wall: GAME OVER appears.
- Fly through the gap: nothing happens.
- Press R: the game restarts.

### Git commit

```
git add .
git commit -m "Task 2: collision and game over"
```

## Task 3: Distance score

### PROMPT SENT TO AI

Now add distance-based scoring to my helicopter game. Track the distance traveled so it increases automatically every frame while the game is running (use the scroll speed, 10 pixels = 1 metre). Show "Distance: N m" at the top-left of the screen. When the game is over, stop increasing it and show "Final distance: N m" under the GAME OVER message. Pressing R to restart must reset the distance to 0. If you need to change `draw_banner` in `renderer.py` to position two lines of text, do that too. Give me the full updated files.

### What the AI changed (game\_engine.py, renderer.py)

- `distance_px` grows by `SCROLL_SPEED` every frame while the game is running.
- `distance` converts pixels to metres: 10 pixels = 1 metre.
- The HUD draws "Distance: N m" at the top-left.
- On game over the distance stops, and "Final distance: N m" shows under GAME OVER.
- `draw_banner` now accepts `offset_y` and `color` so two lines can be placed.
- `reset()` sets `distance_px` back to 0.

### Key code

```
PIXELS_PER_METER = 10

# in reset():
self.distance_px = 0.0
```

```

@property
def distance(self):
    return int(self.distance_px // PIXELS_PER_METER)

# in update():
self.distance_px += SCROLL_SPEED

# in draw():
renderer.draw_text(surface, font, f"Distance: {self.distance} m", (10, 10))
if self.game_over:
    renderer.draw_banner(surface, font, "GAME OVER - press R to restart", offset_y=-20)
    renderer.draw_banner(surface, font, f"Final distance: {self.distance} m",
                        offset_y=20, color=renderer.COLOR_TEXT)

# renderer.py
def draw_banner(surface, font, text, offset_y=0, color=(180, 40, 40)):
    surf = font.render(text, True, color)
    rect = surf.get_rect(center=(surface.get_width() // 2,
                                surface.get_height() // 2 + offset_y))
    surface.blit(surf, rect)

```

### How to check it

- The counter climbs steadily at the top-left.
- At game over the number stops and Final distance shows.
- Press R: distance goes back to 0.

### Git commit

```

git add .
git commit -m "Task 3: distance scoring"

```

## Task 4: Shield

### PROMPT SENT TO AI

Now add a shield mechanic to my helicopter game. Pressing SPACE activates the shield. While active, draw a clearly visible blue circle around the helicopter. The shield protects from exactly one obstacle collision: when the helicopter hits a wall with the shield on, the shield turns off immediately and the game continues. After that the helicopter is vulnerable again. Important: after the shield absorbs a hit, the same obstacle must not kill the player on the next frame (flag that obstacle). Add a 5 second recharge so the shield can't be kept on forever, and show the shield status (READY / ACTIVE / recharging) in the HUD under the distance. Reset the shield on restart. Give me the full updated files.

### What the AI changed (game\_engine.py, obstacle.py, renderer.py)

- SPACE activates the shield, but only if it is not already active and the cooldown is 0.
- While active, a blue circle is drawn around the helicopter.
- On a hit with the shield on: the shield turns off, a 5 second cooldown starts (300 frames), and the game continues.
- The obstacle that was hit gets shield\_absorbed = True, so it cannot kill the player on the next frame.
- The HUD shows Shield: READY, ACTIVE, or recharging with the seconds left.
- reset() clears the shield and cooldown.

### Key code

```

# obstacle.py (__init__)
self.shield_absorbed = False

# game_engine.py
SHIELD_COOLDOWN_FRAMES = 300    # 5 seconds at 60 FPS

def activate_shield(self):

```

```

        if not self.shield_active and self.shield_cooldown == 0:
            self.shield_active = True

# in update(), for each obstacle:
if obstacle.shield_absorbed or not self._hits_obstacle(obstacle):
    continue
if self.shield_active:
    self.shield_active = False
    self.shield_cooldown = SHIELD_COOLDOWN_FRAMES
    obstacle.shield_absorbed = True
else:
    self.game_over = True

# renderer.py: draw_scene(..., shield_active=False)
if shield_active:
    rect = helicopter.get_rect()
    radius = max(rect.width, rect.height) // 2 + 10
    glow = pygame.Surface((radius * 2, radius * 2), pygame.SRCALPHA)
    pygame.draw.circle(glow, (80, 160, 255, 90), (radius, radius), radius)
    pygame.draw.circle(glow, (30, 100, 255, 255), (radius, radius), radius, 3)
    surface.blit(glow, glow.get_rect(center=rect.center))

```

### How to check it

- HUD shows Shield: READY (press SPACE).
- Press SPACE: blue circle appears, HUD shows ACTIVE.
- Hit a wall: game continues, circle disappears, HUD shows recharging.
- Hit another wall right away: GAME OVER.
- After about 5 seconds the HUD shows READY again.
- Press R: everything resets.

### Git commit

```

git add .
git commit -m "Task 4: shield mechanic"

```

## Final steps: videos and submission

- Record before.mp4 (bug visible) and after.mp4 (all fixes and features), about 10 seconds each.
- Copy before.mp4, after.mp4 and chat\_history.pdf into Lab-4/.
- Commit and push: git add . / git commit -m "Lab4: add videos and chat history" / git push origin main.
- Check on GitHub that each task has its own commit. Do not open a PR to SETAPESU26.
- GitHub rejects files over 100 MB, so trim or compress a video that is too big.